

# EventPlanner

## Test Scenario Documentation

### 1. Introduction

This document describes the test scenarios designed for the three most critical functions of the EventPlanner application. Each function is documented with its purpose, input parameters, and expected output across the full range of valid and invalid input combinations. Section 5 adds scenarios for the two methods changed during the correctness and security review, and section 6 contains a complete NUnit test class for the first function as a worked example.

The three functions selected for testing are:

1. `Registration.ComputeStatus` — derives the payment status from price and paid amount.
2. `RegistrationService.CreateRegistration` — enforces event capacity and the no-duplicate-registration rule.
3. `FinanceService.Profit` — calculates the profit of an event from registrations and expenses.

These functions were chosen because they encode core business rules. A defect in any of them would corrupt the financial reporting, allow event overselling, or misclassify participant payments, which are all critical to the correctness of the application.

### 2. Function 1 — `Registration.ComputeStatus`

#### 2.1 Purpose

Pure static function that derives a `PaymentStatus` value from a ticket price, a paid amount, and an optional cancellation flag. It is called whenever a registration is created or its payment values are updated, and it ensures that the stored status is always consistent with the actual numbers.

#### 2.2 Signature

```
public static PaymentStatus ComputeStatus(decimal price, decimal paidAmount, bool cancelled = false)
```

#### 2.3 Decision logic

The function evaluates four rules in this order:

1. If `cancelled` is true, return `Cancelled`.
2. If price is essentially zero ( $\leq 0.005$  €), return `Free`.
3. If paid amount is essentially zero ( $\leq 0.005$  €), return `Unpaid`.
4. If paid amount is less than price (minus tolerance), return `PartPaid`. Otherwise return `Paid`.

A tolerance of 0.005 € is applied around comparisons because price and paid amount are stored as SQLite REAL (double) and round-tripped through `System.Decimal`, which can introduce tiny rounding errors.

#### 2.4 Test scenarios

| Price  | Paid amount | Cancelled | Expected status | Reasoning                                                 |
|--------|-------------|-----------|-----------------|-----------------------------------------------------------|
| 0.00 € | 0.00 €      | false     | Free            | Zero price means a free ticket regardless of paid amount. |

| Price    | Paid amount | Cancelled | Expected status | Reasoning                                                       |
|----------|-------------|-----------|-----------------|-----------------------------------------------------------------|
| 0.00 €   | 0.00 €      | true      | Cancelled       | The cancelled flag always wins.                                 |
| 50.00 €  | 0.00 €      | false     | Unpaid          | Nothing has been paid yet.                                      |
| 50.00 €  | 25.00 €     | false     | PartPaid        | Some money received, but less than the full price.              |
| 50.00 €  | 50.00 €     | false     | Paid            | Full price has been collected.                                  |
| 50.00 €  | 60.00 €     | false     | Paid            | Overpayment is treated as paid (refund logic is outside scope). |
| 50.00 €  | 50.00 €     | true      | Cancelled       | Cancelled wins even if fully paid; refund handled separately.   |
| 50.00 €  | 0.003 €     | false     | Unpaid          | Paid amount below 0.005 € tolerance counts as zero.             |
| 50.00 €  | 49.998 €    | false     | Paid            | Paid amount within 0.005 € of price counts as fully paid.       |
| 0.004 €  | 0.00 €      | false     | Free            | Price below 0.005 € tolerance counts as free.                   |
| 100.00 € | 100.00 €    | true      | Cancelled       | Cancelled flag overrides the Paid result.                       |

### 3. Function 2 — RegistrationService.CreateRegistration

#### 3.1 Purpose

Creates a validated Registration object for the given event and person. Before constructing the object, it enforces two business rules that the schema alone cannot guarantee: the event must not exceed its capacity, and a person cannot be registered for the same event twice (unless their previous registration was cancelled).

#### 3.2 Signature

```
public Registration CreateRegistration(Event selectedEvent, int personId, string
ticketType, decimal price, decimal paidAmount)
```

#### 3.3 Decision logic

1. Count active (non-cancelled) registrations for the event. If the count is  $\geq$  capacity, throw `InvalidOperationException`.
2. Check whether the person already has an active registration for the event. If yes, throw `InvalidOperationException`.
3. Build a new Registration, call `UpdatePayment` to compute the status, then run domain validation. Throw `ArgumentException` with the aggregated error messages if validation fails.

#### 3.4 Test scenarios

| Input scenario                                  | Expected output                   | Reasoning                                                                |
|-------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------|
| Capacity 10, 4 active registrations, new person | Returns a new Registration object | Capacity check passes ( $4 < 10$ ) and the person is not yet registered. |

| Input scenario                                                               | Expected output                               | Reasoning                                                                                             |
|------------------------------------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Capacity 10, 10 active registrations, new person                             | Throws <code>InvalidOperationException</code> | Capacity has been reached. Message: 'Event capacity has been reached.'                                |
| Capacity 10, 5 active registrations + 3 cancelled, new person                | Returns a new Registration                    | Cancelled registrations do not count towards capacity.                                                |
| Capacity 10, 4 registrations, same person already registered (not cancelled) | Throws <code>InvalidOperationException</code> | Duplicate registration rejected. Message: 'This person is already registered for the selected event.' |
| Capacity 10, 4 registrations, same person was previously cancelled           | Returns a new Registration                    | Cancelled past registrations do not block a fresh registration.                                       |
| Price 50 €, paid 50 €, valid event and person                                | Returns a Paid registration                   | Validates successfully and status is computed as Paid.                                                |
| Price 0 €, paid 0 €, valid event and person                                  | Returns a Free registration                   | Free event, zero amounts are accepted.                                                                |
| Price 50 €, paid 75 € (paid > price)                                         | Throws <code>ArgumentException</code>         | Validation rejects paid > price.                                                                      |
| Price 0 €, paid 10 € (free ticket with money)                                | Throws <code>ArgumentException</code>         | Free ticket cannot have a non-zero paid amount.                                                       |
| Price -5 €                                                                   | Throws <code>ArgumentException</code>         | Validation rejects negative price.                                                                    |
| PersonId = 0 (no person selected)                                            | Throws <code>ArgumentException</code>         | Validation rejects missing person.                                                                    |

## 4. Function 3 — FinanceService.Profit

### 4.1 Purpose

Computes the profit of an event by subtracting total expenses from paid income across a collection of registrations and a collection of expenses. The function only counts money actually received (paid income), not expected income, and it ignores cancelled registrations entirely. Drives the Finance dashboard card and the finance summary CSV export.

### 4.2 Signature

```
public static decimal Profit(IEnumerable<Registration> regs, IEnumerable<Expense> expenses)
```

### 4.3 Decision logic

1. Sum the PaidAmount of every registration whose status is not Cancelled.
2. Sum the Amount of every expense.
3. Return paid income minus total expenses. The result can be negative.

### 4.4 Test scenarios

| Input scenario                | Expected output | Reasoning                                |
|-------------------------------|-----------------|------------------------------------------|
| No registrations, no expenses | 0.00 €          | Profit = paid income (0) – expenses (0). |
| 1 paid 50 €, no expenses      | 50.00 €         | Pure income.                             |
| 1 paid 50 €, expenses 30 €    | 20.00 €         | Straightforward profit.                  |

| Input scenario                                          | Expected output | Reasoning                                                                   |
|---------------------------------------------------------|-----------------|-----------------------------------------------------------------------------|
| 1 paid 50 €, 1 unpaid 50 €, expenses 0 €                | 50.00 €         | Unpaid amount does not contribute to profit (only paid income does).        |
| 1 partly paid (50 € price, 30 € paid), expenses 10 €    | 20.00 €         | Only the 30 € collected counts; expenses subtracted.                        |
| 1 paid 80 €, 1 cancelled (was 50 € paid), expenses 20 € | 60.00 €         | Cancelled registrations excluded from income even if they had paid amounts. |
| 3 free (price 0, paid 0), expenses 30 €                 | -30.00 €        | Free event with running costs gives a loss.                                 |
| 2 paid 50 €, expenses 200 €                             | -100.00 €       | Loss when expenses exceed income.                                           |
| 1 paid 99.99 €, expenses 50.00 €                        | 49.99 €         | Decimal precision is preserved.                                             |

## 5. Review scenarios (defect fixes)

The following scenarios were added after a correctness and security review of the code base. They cover the two methods that were changed as a result of that review: the registration reactivation logic and the CSV escaping routine. Both fixes are described in section 5.3 of the Final Report.

### 5.1 RegistrationRepository.AddOrReactivate

This method inserts a new registration, or, when a row already exists for the same event and person, reuses and reactivates that row instead of inserting a duplicate that would violate `UNIQUE(EventId, PersonId)`.

| Input scenario                                              | Expected output                                                                         | Reasoning                                                                                                               |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| New person, no existing row for this event                  | Inserts a new row and returns its new id                                                | No existing (EventId, PersonId) row, so the normal insert path runs.                                                    |
| Person was registered, then cancelled, now registered again | Reuses the existing row, reactivates it, returns the same id; no duplicate and no error | The cancelled row still holds the UNIQUE slot, so it is updated instead of inserted. This is the defect that was fixed. |
| Person already has an active (non-cancelled) registration   | Blocked earlier by RegistrationService; AddOrReactivate is never reached                | The business rule rejects active duplicates first, so reactivation only ever applies to cancelled rows.                 |

### 5.2 CsvExportService.Escape (formula-injection guard)

Escape now prefixes any value that begins with `=`, `+`, `-`, `@`, tab or carriage return with a single apostrophe, then applies the existing RFC 4180 quoting. The output column shows the exact text written to the CSV file.

| Input value | Escaped output | Reasoning                                    |
|-------------|----------------|----------------------------------------------|
| Standard    | Standard       | No leading formula character and no comma or |

| Input value         | Escaped output        | Reasoning                                                                                     |
|---------------------|-----------------------|-----------------------------------------------------------------------------------------------|
|                     |                       | quote, so it is returned unchanged.                                                           |
| =2+5                | '=2+5                 | Leading = would be evaluated as a formula; the apostrophe forces the cell to be literal text. |
| @SUM(A1:A9)         | '@SUM(A1:A9)          | Leading @ also starts a formula in some spreadsheets, so it is neutralised the same way.      |
| Brand, storytelling | "Brand, storytelling" | No formula character, but the comma still triggers the standard RFC 4180 quoting.             |

## 6. Unit test code

The unit test below covers Function 1 (Registration.ComputeStatus). It uses NUnit and the TestCase attribute, which allows a single test method to exercise many input combinations cleanly. The test cases mirror the scenarios in section 2.4. Two additional [Test] methods cover related behaviour: UpdatePayment recomputing the status, and CheckIn refusing cancelled registrations.

To run the tests, add a separate NUnit test project to the solution that references the main EventPlanner project, install the NUnit and NUnit3TestAdapter NuGet packages, drop this file into it, and run the tests through Test Explorer or the dotnet test command.

### 6.1 File: RegistrationComputeStatusTests.cs

```
using EventPlanner.Domain;
using NUnit.Framework;

namespace EventPlanner.Tests;

[TestFixture]
public class RegistrationComputeStatusTests
{
    // Free: when price is zero (or within tolerance), regardless of paid amount.
    [TestCase(0.0, 0.0, false, ExpectedResult = PaymentStatus.Free)]
    [TestCase(0.004, 0.0, false, ExpectedResult = PaymentStatus.Free)]

    // Unpaid: positive price, no payment received.
    [TestCase(50.0, 0.0, false, ExpectedResult = PaymentStatus.Unpaid)]
    [TestCase(50.0, 0.003, false, ExpectedResult = PaymentStatus.Unpaid)]

    // PartPaid: some money received but less than price.
    [TestCase(50.0, 25.0, false, ExpectedResult = PaymentStatus.PartPaid)]
    [TestCase(100.0, 99.9, false, ExpectedResult = PaymentStatus.PartPaid)]

    // Paid: full price received (or overpaid, or within tolerance).
    [TestCase(50.0, 50.0, false, ExpectedResult = PaymentStatus.Paid)]
    [TestCase(50.0, 49.998, false, ExpectedResult = PaymentStatus.Paid)]
    [TestCase(50.0, 60.0, false, ExpectedResult = PaymentStatus.Paid)]
}
```

```

// Cancelled: the flag wins in every scenario.
[TestCase(0.0, 0.0, true, ExpectedResult = PaymentStatus.Cancelled)]
[TestCase(50.0, 0.0, true, ExpectedResult = PaymentStatus.Cancelled)]
[TestCase(50.0, 50.0, true, ExpectedResult = PaymentStatus.Cancelled)]
public PaymentStatus ComputeStatus_Returns_Expected(double price, double paidAmount,
bool cancelled)
{
    return Registration.ComputeStatus((decimal)price, (decimal)paidAmount, cancelled);
}

[Test]
public void UpdatePayment_Recomputes_Status_From_Fresh_Values()
{
    var registration = new Registration { EventId = 1, PersonId = 1 };

    registration.UpdatePayment(100m, 0m);
    Assert.That(registration.Status, Is.EqualTo(PaymentStatus.Unpaid));

    registration.UpdatePayment(100m, 40m);
    Assert.That(registration.Status, Is.EqualTo(PaymentStatus.PartPaid));

    registration.UpdatePayment(100m, 100m);
    Assert.That(registration.Status, Is.EqualTo(PaymentStatus.Paid));

    registration.UpdatePayment(100m, 100m, cancelled: true);
    Assert.That(registration.Status, Is.EqualTo(PaymentStatus.Cancelled));
}

[Test]
public void CheckIn_Throws_For_Cancelled_Registration()
{
    var registration = new Registration { EventId = 1, PersonId = 1 };
    registration.UpdatePayment(50m, 50m, cancelled: true);

    Assert.Throws<InvalidOperationException>(() => registration.CheckIn());
}
}

```

## 7. Summary

Three critical functions were selected for systematic test scenario design. Each function was analysed for the full range of input combinations, including edge cases (zero values, rounding tolerance, cancellation overrides, capacity boundaries, duplicate detection). A working NUnit test class is provided for the most critical function. The same pattern (parameterised TestCase attributes for pure functions, [Test] methods for object behaviour) can be applied to extend the test suite to the remaining two functions and to other parts of the system.